

Byzantine Agreement

```
#include<stdio.h>
#include<mpi.h>
void main(int argc,char* argv[])
{
int rank,size,i,j,r,k,count,faulty=0,majorityValue;

MPI_Init(&argc,&argv);
MPI_Comm_size(MPI_COMM_WORLD,&size);
MPI_Comm_rank(MPI_COMM_WORLD,&rank);

MPI_Status st;

if(rank==2||rank==5)
faulty=1;

if(faulty==1)
printf("i am rank %d Faulty..find me \n",rank);

int recv[size];
if(faulty!=1)
{
for(i=0;i<size;i++)
{
if(i!=rank)
{
MPI_Send(&rank,1,MPI_INT,i,101,MPI_COMM_WORLD);
}
}
for(i=0;i<size;i++)
{
if(i==rank)
recv[i]=rank;
else
{
MPI_Recv(&recv[i],1,MPI_INT,i,101,MPI_COMM_WORLD,&st);
}
}
}
else if(faulty==1)
{
for(i=0;i<size;i++)
{
if(i!=rank)
{
r=(rand()+r*r)%100;
MPI_Send(&r,1,MPI_INT,i,101,MPI_COMM_WORLD);
}
}
```

```

    }
    for(i=0;i<size;i++)
    {
        if(i==rank)
            recv[i]=rank;
        else
        {
            MPI_Recv(&recv[i],1,MPI_INT,i,101,MPI_COMM_WORLD,&st);
        }
    }
    for(i=0;i<size;i++)
        recv[i]=(rand()+r*r)%100;
}

for(i=0;i<size;i++)
    if(i!=rank)
        MPI_Send(recv,size,MPI_INT,i,201,MPI_COMM_WORLD);

int vect[size][size];
for(i=0;i<size-1;i++)
    MPI_Recv(vect[i],size,MPI_INT,MPI_ANY_SOURCE,201,MPI_COMM_WORLD,&st);

for(i=0;i<size;i++)
{
    for(j=0;j<size-1;j++)
    {
        count=0;
        majorityValue=vect[j][i];
        for(k=0;k<size-1;k++)
        {
            if(vect[k][i]==majorityValue)
                count++;
            else
                count--;
        }
        if(count>0)
        {
            break;
        }
    }
    if(j==size-1 && faulty!=1)
        printf("%d says %d is Faulty\n",rank,i);
}
MPI_Finalize();
}

```

Output

```

mpicc byzantine.c -o byz
mpirun -np 6 byz

```

i am rank 2 Faulty..find me

i am rank 5 Faulty..find me

0 says 2 is Faulty

0 says 5 is Faulty

3 says 2 is Faulty

1 says 2 is Faulty

3 says 5 is Faulty

1 says 5 is Faulty

4 says 2 is Faulty

4 says 5 is Faulty